Running a specialist on a story

For the developer who just picked up a story in **To-Do**.

As of 2026-08-07, every surface is app-dispatched — moving a story to `In-Process` starts a real `dispatch-worker` workflow against the specialist sandbox, no manual steps needed. This document is now a manual fallback: useful for debugging a dispatch, running against a target repo the app-dispatch path doesn't reach yet, or working without the app's infrastructure at all. It is no longer any specialist type's primary path.

You do three things: set the branch up, paste a prompt, review what comes back. Everything below is the detail behind those three.

First time only

1. Claude Code, from Claude Desktop, pointed at the workspace folder

Start a Claude Code session in Claude Desktop and point it at the **workspace folder** — the parent directory holding all the repos — not at a single repo. The specialist needs to read files, run git, run the tests, and open a pull request, and it needs more than one repo in view to do it.

2. Lay the workspace out

```
~/work/<project>/                ← point Claude Code here
├── <surface-a>/                 ← a product repo (frontend, backend, …)
├── <surface-b>/                 ← another product repo
└── intent-to-production/        ← the framework repo (agent definitions)
```

Everything in one place because reading across repos matters: a frontend story has to check the API the backend actually shipped, not the one the story describes.

Writes go to one repo only. Every story targets a single surface. That repo is where the branch lives, where the code lands, and where the PR opens. The prompt names it. The others are read-only.

3. Clone `intent-to-production`

The specialist definitions live there, not in the product repos. Note the absolute path — the prompt points at it.

`git pull` before each run. A stale clone runs an out-of-date definition and you won't notice until the output is wrong.

4. Turn on the connectors

- **Linear** — the specialist reads the story and writes its report through it. Without it the run can't report anything.
- **Source control** — an authenticated `gh`, or the GitHub connector, so it can open the PR.

Git itself runs locally in your checkout.

5. Approve tool calls as they come

Claude will ask before running git, the test suite, or `gh pr create`. Approve them one at a time rather than pre-approving in `.claude/settings.json`. For the first few stories you want to see what it actually does.

Every story

- ☐ Story read, and it makes sense to you
- ☐ Your questions asked and answered **in the story's comment thread**
- ☐ Branch chain set up in the target surface's repo, story branch checked out
- ☐ `git pull` in the framework clone
- ☐ Claude Code session pointed at the workspace folder

Read the story

You're going to review whatever comes back. If it doesn't make sense now, it won't make more sense as a diff.

Ask your questions in the story's comment thread

Not in Slack, not in a call. The specialist reads that thread as part of the story — anything the architect answers there, it picks up. Anything answered anywhere else, it never sees.

Ask before you dispatch, not during review.

Set the branch chain up

```
main
└── <BRD branch>              one per project
    └── <epic branch>          the tracker's branch name on the epic
        └── <story branch>     the tracker's branch name on the story
```

Use the branch names the tracker already assigns to the epic and story — don't make up your own. The BRD branch is whatever the project's branch is called; epic branches come off it, never off `main`.

Check out the story branch before you dispatch.

The specialist checks this chain and stops if it's wrong. It won't create or re-parent a branch for you.

A cross-story test story is cut after the stories it tests. Create its branch once those have merged into the epic branch — not after every story in the epic, just the ones its coverage needs. Cut it early and the code it exercises isn't underneath it yet.

The prompt

There is one specialist definition now. The story's `surface:` label says where it works, not which file to load.

```
Read these three files now — they define your role and the contracts you work
to:

- <FRAMEWORK_PATH>/agents/specialist.md
- <FRAMEWORK_PATH>/skills/story-contract/SKILL.md
- <FRAMEWORK_PATH>/skills/epic-writing/SKILL.md

You are the Specialist those files describe. Follow that definition; this
message only tells you which story and where.

Assignment: story <STORY_ID> — "<STORY_TITLE>", under epic <EPIC_ID>.

Your target surface is ./<SURFACE_REPO>, per the story's `surface:` label(s).
Every write you make goes there and nowhere else. It is checked out on <STORY_BRANCH>; the epic branc
<EPIC_BRANCH>. Both names come from the tracker, and I set the chain up before
dispatching you — verify it, do not repair it.

The other repositories in this workspace are sibling surfaces. Read them when
you need to confirm what a dependency actually implements rather than what the
story claims it does. Do not modify them and do not open a PR against them.

Using the Linear connector, read <STORY_ID>'s description and its full comment
thread, then walk up to <EPIC_ID> for the parent epic, its resolved API map
(both sections), its attached design evidence, and the project's design issue
for cross-cutting rules. The comment thread on <STORY_ID> carries a
question-and-answer exchange between me and the architect from before you were
engaged — read it as part of the story, not as commentary on it.

Then act per your definition: check blocking dependencies, verify the branch
chain, read the codebase and its conventions spec, implement, run the tests,
open the PR into <EPIC_BRANCH>, and post your completion report on
<STORY_ID>.

If the branch chain is wrong, a blocking dependency is unmerged, or the story
has a gap you cannot resolve from the thread — stop and report it rather than
deciding for yourself. A blocker you surface is the useful output of this run.
```

Placeholder	Where it comes from
`<FRAMEWORK_PATH>`	Absolute path to your `intent-to-production` clone
`<STORY_ID>`, `<STORY_TITLE>`	The story
`<EPIC_ID>`	The story's parent epic
`<SURFACE_REPO>`	Folder name of the target repo in your workspace
`<STORY_BRANCH>`, `<EPIC_BRANCH>`	The branch names on the story and the epic

What comes back

A PR from the story branch into the epic branch, and a comment on the story reporting one of three outcomes. No label — the comment is the only record:

Outcome	Meaning
Complete	Done. PR is open, tests pass.
Waiting	A story it depends on isn't merged yet. Nothing was written.
Blocked	It hit something it wouldn't guess at — a gap in the story, a wrong branch base, a conflict. Read the comment; it names the specific thing.

CI runs the tests on the PR. A developer other than you reads the diff and merges it into the epic branch. No agent reviews it.

Read the "Questions & assumptions" section of the report. Each entry is a place the story was vague enough that the agent had to decide something. Those are worth passing back to whoever wrote it.